

B4演習 2026 : EX2 レポート

加藤稔之典

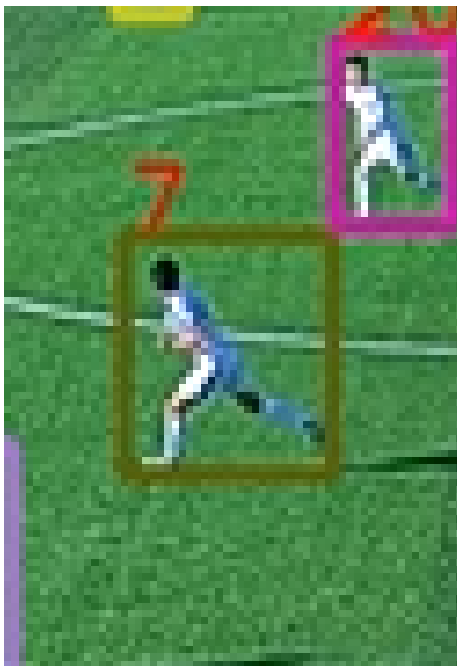
2025年5月15日

1 はじめに

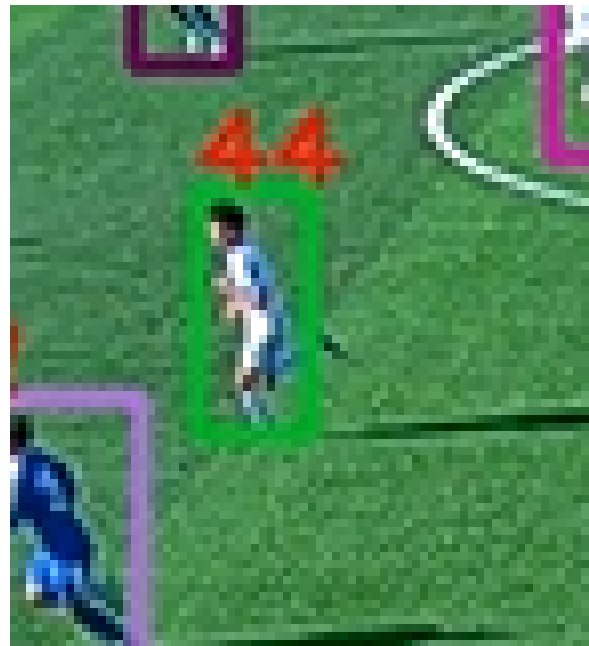
本課題では、マルチオブジェクトトラッキング (MOT) アルゴリズムである「Deep-ElIoU」を用い、サッカー動画における選手の追跡および動特性の分析を行う。

事前準備として、SoccerTrack Challenge 2025の動画 (117093.mp4) の先頭20秒間を切り出した「117093_trimmed.mp4」に対してDeep-ElIoUを実行した。実行結果の動画 (https://youtu.be/PTTbHZr-GTg?si=8rTQC4j-_cTu57fs) を定性的に評価すると、選手が密集するシーンや交差するシーンにおいて追跡が不安定になる傾向が見られた。

図1および図2に、同一選手に対してIDが変化 (IDスイッチ) している代表的な事例を示す。

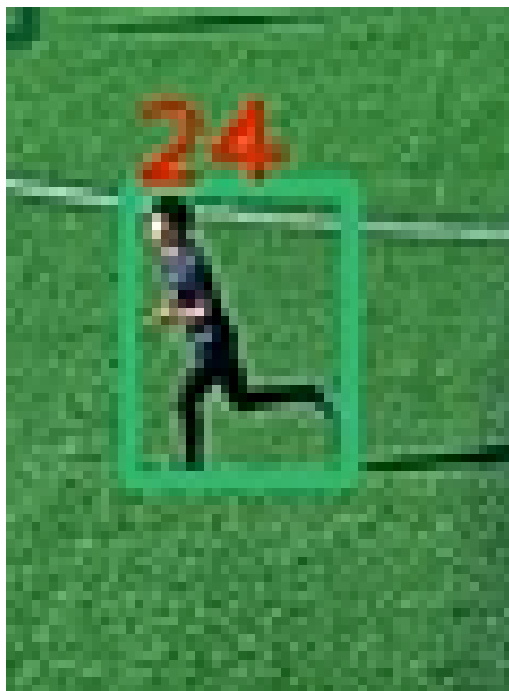


(a) 動画2秒時点 (ID: 7)



(b) 動画3秒時点 (ID: 44へ変化)

図1: IDスイッチの事例① (同一選手が短時間で異なるIDとして認識されている)



(a) 動画5秒時点 (ID: 24)



(b) 動画6秒時点 (ID: 7へ変化)

図2: IDスイッチの事例② (消失後の再検知によるIDの入れ替わり)

2 トラッキングデータの構成

Deep-EIoUが出力するMOTChallenge形式のデータ構造をプログラムコード1に示す。

`<frame>, <id>, <x>, <y>, <w>, <h>, <score>, <v1>, <v2>, <v3>`

本解析では、各バウンディングボックスの中心座標 (cx, cy) を $cx = x + w/2, cy = y + h/2$ として算出し、解析に用いた。

```
1 0,1,2005.74,435.93,53.52,117.64,7.43,-1,-1,-1
2 0,2,1740.38,316.08,33.66,82.09,7.33,-1,-1,-1
3 ... (中略) ...
```

プログラムコード 1: トラッキング結果txtファイル (一部抜粋)

3 課題1: 選手軌跡の可視化

特定の選手ID (7, 24, 44: IDスイッチ発生群、13: ゴールキーパー) を抽出し、画像平面上の軌跡をプロ

ットした。プログラムをコード2に、結果を図3に示す。

```
1  # 課題1: 選手軌跡の可視化
2  import pandas as pd
3  import matplotlib.pyplot as plt
4
5
6  file_path =
   ↳  "/content/Deep-ElIoU/Deep-ElIoU/YOLOX_outputs/yolox_x_ch_sportsmot/track_vis/2026_05_15_04_18_46.txt"
7
8  cols = ['frame', 'id', 'x', 'y', 'w', 'h', 'score', 'v1', 'v2', 'v3']
9  df = pd.read_csv(file_path, header=None, names=cols)
10
11 # 中心座標の計算
12 df['cx'] = df['x'] + df['w'] / 2
13 df['cy'] = df['y'] + df['h'] / 2
14
15 # 特定のIDをピックアップして描画
16 target_ids = [7, 13, 24, 44]
17 plt.figure(figsize=(10, 6))
18 for tid in target_ids:
19     target_df = df[df['id'] == tid].sort_values('frame')
20     if not target_df.empty: # IDが存在する場合のみプロット
21         plt.plot(target_df['cx'], target_df['cy'], marker='o', markersize=2, label=f'ID
   ↳  {tid}')
22
23 plt.title('Player Trajectories')
24 plt.xlabel('x [px]')
25 plt.ylabel('y [px]')
26 plt.gca().invert_yaxis()
27 plt.legend()
28 plt.grid(True)
29 plt.show()
```

プログラムコード 2: 選手軌跡の可視化プログラム

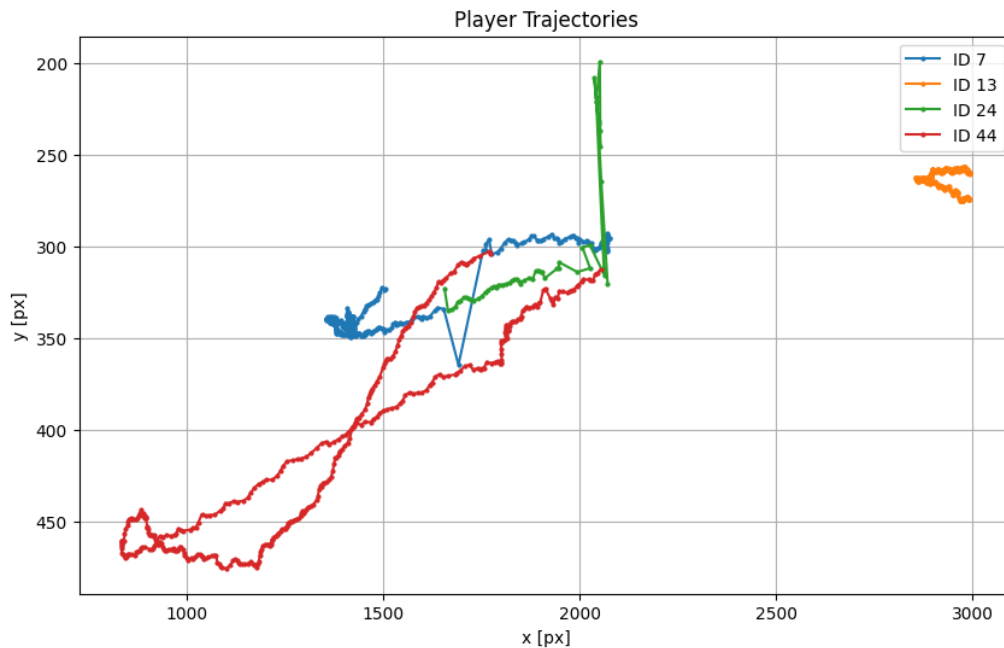


図3: 選手軌跡の可視化結果

分析結果：図3の可視化結果から、以下の現象が確認できる。

- 不連続な開始点 (ID 44)：ID 44の軌跡（赤色）がフィールド中央付近から突如として開始している。これは、同一選手がそれまで別のID（ID 7等）で追跡されていたものが、遮蔽後に新規IDとして割り当てられた「IDスイッチ」の典型的な証拠である。
- 軌跡の途切れ (ID 24)：ID 24（緑色）は短い移動の後に消失している。図2の動画キャプチャと照らし合わせると、選手同士の交差によるオクルージョン（遮蔽）が発生し、検出器が対象を見失った（False Negative）ことが原因である。
- 安定した軌跡 (ID 13)：ゴールキーパー（ID 13）は、移動範囲が狭く周囲との交差も少ないため、途切れのない滑らかな軌跡を描いている。

4 課題2：動特性分析（速度・加速度）

フレーム間の中心座標の差分から算出した速度の推移をプログラムコード3および図4に示す。

```

1  # 課題2: 速度・加速度を用いた分析
2  import numpy as np
3
4  plt.figure(figsize=(15, 10))
5
6  for i, tid in enumerate(target_ids):
7      # 特定のIDを抽出してフレーム順にソート
8      p_df = df[df['id'] == tid].sort_values('frame').copy()
9
10     if p_df.empty:
11         print(f"ID {tid} はデータ内に見つかりません。")
12         continue
13
14     # 1. 速度 (speed) の計算
15     # diff() は現在の行と前の行の差分を計算します
16     p_df['vx'] = p_df['cx'].diff()
17     p_df['vy'] = p_df['cy'].diff()
18     p_df['speed'] = np.sqrt(p_df['vx']**2 + p_df['vy']**2)
19
20     # 2. 加速度 (acceleration) の計算
21     p_df['accel'] = p_df['speed'].diff()
22
23
24
25     # グラフ描画
26     plt.subplot(2, 2, i+1)
27     plt.plot(p_df['frame'], p_df['speed'], label='Speed')
28
29
30     plt.title(f'Tracking Analysis: ID {tid}')
31     plt.xlabel('Frame')
32     plt.ylabel('Speed [px/frame]')
33     plt.legend()
34     plt.grid(True, alpha=0.3)
35
36
37 plt.tight_layout()
38 plt.show()

```

プログラムコード 3: 速度・加速度を用いた分析プログラム

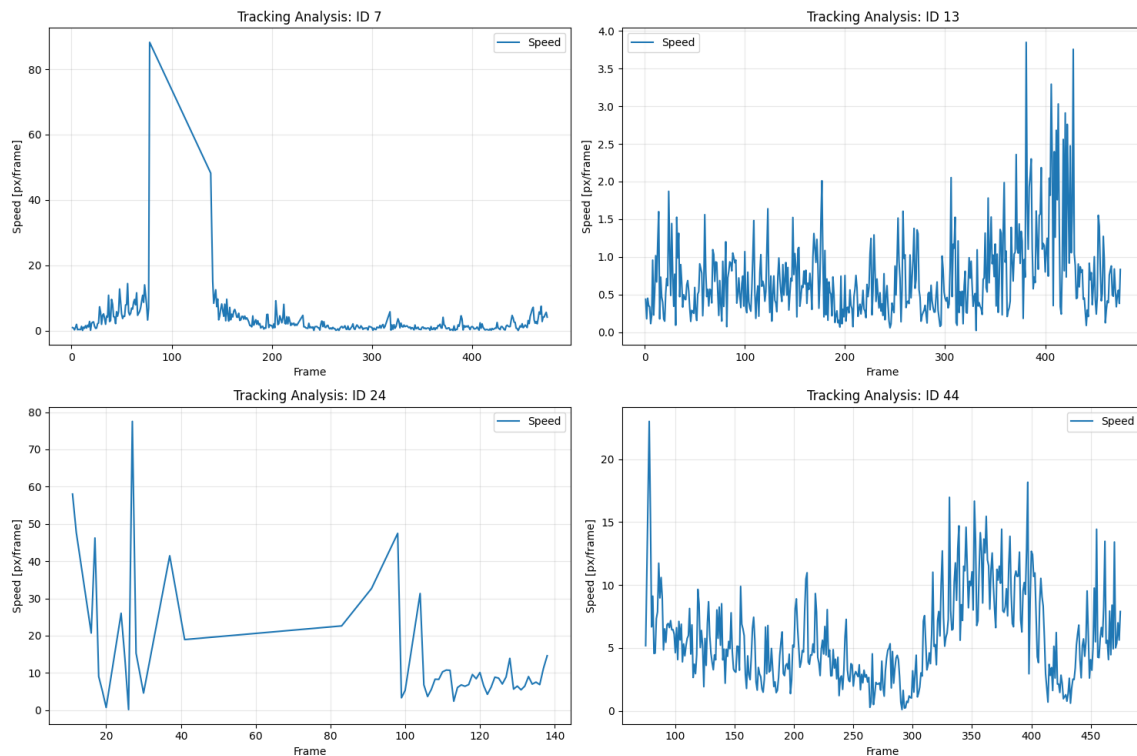


図4: 対象IDの速度推移グラフ

考察：速度グラフの時系列変化を分析した結果、トラッキングの質に関して以下の知見が得られた。

1. 物理的限界を超えるスパイク (ID 7, 44) : ID 7のFrame 70付近、およびID 44の出現直後において、速度が **60 px/frame** 以上に達する急激なスパイクが確認された。これはサッカー選手の画像平面上の最大移動速度を大きく逸脱しており、IDが他選手へ飛び移った (IDスイッチ)、あるいは検出ボックスが急激にリサイズされたことによるノイズである。
2. 追跡失敗によるデータの空白: ID 24およびID 44のグラフにおいて、データがプロットされていない「空白期間」が存在する。これは検出スコアが低下し、Deep-ElIoUが追跡を断念した期間と一致する。特にID 44は、ID 7が消失したタイミングと入れ替わりで出現しており、同一人物に対する追跡の断絶を数値的に裏付けている。
3. 検出ノイズの特性: 低速時 (ID 13等) に見られる細かな振動は、検出器 (YOLOX) が出力するバウンディングボックスの「揺れ」を反映している。これらは移動平均フィルタ等の後処理 (課題3の発展内容) によって平滑化の余地がある。